

Questão 1

```
WITH alugueis_mes AS (  
  SELECT  
    YEAR(rental.rental_date) AS ano,  
    MONTH(rental.rental_date) AS mes,  
    inventory.film_id,  
    film.title,  
    COUNT(*) AS total_aluguel_filme  
  
  FROM rental  
  JOIN inventory  
    ON rental.inventory_id = inventory.inventory_id  
  JOIN film  
    ON inventory.film_id = film.film_id  
  
  GROUP BY  
    YEAR(rental.rental_date),  
    MONTH(rental.rental_date),  
    inventory.film_id,  
    film.title  
)  
  
ranking_filmes AS (  
  SELECT  
    ano,  
    mes,  
    film_id,  
    title,  
    total_aluguel_filme,  
  
    DENSE_RANK() OVER (  
      PARTITION BY ano, mes  
      ORDER BY total_aluguel_filme DESC  
    ) AS ranking,  
  
    SUM(total_aluguel_filme) OVER (  
      PARTITION BY ano  
      ORDER BY mes  
    ) AS acumulado_alugueis  
  
  FROM alugueis_mes  
)  
  
SELECT  
  ano,
```

```
    mes,  
    film_id,  
    title,  
    total_aluguel_filme,  
    ranking,  
    acumulado_alugueis  
  
FROM ranking_filmes  
  
WHERE ranking <= 3  
  
ORDER BY  
    ano ASC,  
    mes ASC,  
    ranking ASC,  
    total_aluguel_filme DESC;
```

Explicação:

Primeiro Bloco — CTE `alugueis_mes`

A CTE `alugueis_mes` serve como base da consulta. Nela são extraídos o ano e o mês da data do aluguel, identificado o filme por meio dos `JOINS` entre as tabelas `rental`, `inventory` e `film`, e contabilizada a quantidade de aluguéis de cada filme em cada mês. O `GROUP BY` agrupa os registros por ano, mês e filme, permitindo que o `COUNT(*)` contabilize a quantidade de aluguéis de cada grupo.

Segundo Bloco — CTE `ranking_filmes`

O `PARTITION BY ano, mes` divide os registros em partições independentes para cada combinação de ano e mês. Dessa forma, o ranking é reiniciado a cada mês de cada ano, evitando que meses iguais de anos diferentes participem do mesmo ranking.

O `ORDER BY` do `DENSE_RANK()` ordena `total_aluguel_filme` de forma decrescente. Dessa maneira, os filmes que tiveram a maior quantidade de aluguéis no mês recebem as primeiras posições do ranking.

O `SUM(total_aluguel_filme) OVER (PARTITION BY ano ORDER BY mes)` é utilizado para calcular o acumulado dos aluguéis ao longo dos meses. Dessa forma, o acumulado considera a quantidade de aluguéis dos meses anteriores juntamente com a quantidade de aluguéis do mês atual. Esse processo continua até o último mês do ano e, quando começa um novo ano, o acumulado é reiniciado devido ao `PARTITION BY ano`.

Terceiro Bloco — `SELECT` final

O terceiro bloco utiliza os dados gerados pela segunda CTE e define quais informações serão exibidas no resultado final.

O **WHERE ranking <= 3** é utilizado para mostrar somente os filmes que estão nas posições de 1 a 3 do ranking de cada mês.

O **ORDER BY** serve para organizar o resultado, começando pelo menor ano, depois pelos meses em ordem crescente, pelo ranking de 1 a 3 e, por fim, pela quantidade de alugueis do filme em ordem decrescente. Dessa forma, os dados ficam organizados cronologicamente e de acordo com a posição dos filmes no ranking.

Partição

O **PARTITION BY ano, mes** foi utilizado para que o ranking dos filmes fosse calculado separadamente para cada mês de cada ano. Dessa forma, o ranking é reiniciado a cada novo mês e também é separado entre anos diferentes.

Nível temporal

Como eu desejava analisar os dados de forma mensal, a data do aluguel foi decomposta em ano e mês utilizando **YEAR()** e **MONTH()**. Esses dois níveis temporais permitem calcular o ranking separadamente para cada mês de cada ano.

Medida

A medida utilizada é a quantidade de alugueis, representada por **COUNT(*)**. A partir dela, é utilizado o **SUM(total_aluguel_filme) OVER (PARTITION BY ano ORDER BY mes)** para calcular o acumulado dos alugueis ao longo dos meses.

Dessa forma, o acumulado considera a quantidade de alugueis dos meses anteriores juntamente com a quantidade de alugueis do mês atual. Esse processo continua até o último mês do ano. Quando um novo ano começa, o acumulado é reiniciado devido ao **PARTITION BY ano**.

Questão 2

```
CREATE OR REPLACE VIEW vw_clientes_destaque AS
```

```
WITH gastos_mensais AS (  
  SELECT  
    p.customer_id AS id_cliente,  
    DATE_FORMAT(p.payment_date, '%Y-%m') AS mes_referencia,  
    SUM(p.amount) AS valor_total_pago  
  FROM payment p  
  GROUP BY  
    p.customer_id,
```

```
DATE_FORMAT(p.payment_date, '%Y-%m')
),
```

```
media_mensal AS (
  SELECT
    mes_referencia,
    AVG(valor_total_pago) AS media_clientes
  FROM gastos_mensais
  GROUP BY mes_referencia
),
```

```
comparacao AS (
  SELECT
    gm.id_cliente,
    gm.mes_referencia,
    gm.valor_total_pago,
    mm.media_clientes,

    ROUND(
      (gm.valor_total_pago / mm.media_clientes) * 100,
      2
    ) AS percentual_da_media

  FROM gastos_mensais gm

  INNER JOIN media_mensal mm
    ON gm.mes_referencia = mm.mes_referencia
),
```

```
priorizacao AS (
  SELECT
    id_cliente,
    mes_referencia,
    valor_total_pago,
    media_clientes,
    percentual_da_media,

    CASE
      WHEN percentual_da_media >= 150 THEN 1
      WHEN percentual_da_media >= 125 THEN 2
      WHEN percentual_da_media >= 100 THEN 3
      ELSE NULL
    END AS prioridade

  FROM comparacao
)
```

```
SELECT
```

```

p.id_cliente,

CONCAT(
    c.first_name,
    '',
    c.last_name
) AS nome_cliente,

p.mes_referencia,

ROUND(p.valor_total_pago, 2) AS valor_total_pago,

ROUND(p.media_clientes, 2) AS media_clientes,

p.percentual_da_media,

p.prioridade,

ROW_NUMBER() OVER (
    PARTITION BY p.mes_referencia
    ORDER BY
        p.prioridade ASC,
        p.percentual_da_media DESC,
        p.valor_total_pago DESC
) AS ranking

FROM priorizacao p

INNER JOIN customer c
    ON c.customer_id = p.id_cliente

WHERE p.prioridade IS NOT NULL;

```

Explicação

1. **gastos_mensais**

Agrupa os pagamentos por cliente e por mês. O **SUM(p.amount)** calcula quanto cada cliente gastou naquele mês.

2. **media_mensal**

Calcula a média de gastos dos clientes em cada mês utilizando **AVG()**. A média é separada por mês para que os clientes sejam comparados somente com os demais clientes daquele mesmo período.

3. comparacao

Compara o gasto de cada cliente com a média do mês. O cálculo:

$\text{valor do cliente} \div \text{média} \times 100$

gera o `percentual_da_media`.

Por exemplo, se a média for R\$ 10 e o cliente gastar R\$ 15, ele terá **150% da média**.

4. priorizacao

O `CASE` transforma o percentual em um score:

- 1: gasto $\geq$ 150% da média;
- 2: gasto entre 125% e 150%;
- 3: gasto entre 100% e 125%;
- abaixo de 100%: não é considerado uma ocorrência de destaque.

5. SELECT final

Busca o nome do cliente na tabela `customer`, arredonda os valores e apresenta as informações finais da View.

6. ROW_NUMBER()

Cria um ranking separado para cada mês (`PARTITION BY`). Primeiro são considerados os clientes de maior prioridade e, dentro da mesma prioridade, aqueles com maior percentual da média.

7. WHERE p.prioridade IS NOT NULL

Remove os clientes que ficaram abaixo da média, deixando na View somente as ocorrências que atendem à regra de negócio.

Questão 3

```
CREATE TABLE rental_audit_log (  
    audit_id INT NOT NULL AUTO_INCREMENT,  
    event_timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    db_user VARCHAR(255) NOT NULL,  
    operation VARCHAR(20) NOT NULL,  
    customer_id SMALLINT UNSIGNED NOT NULL,  
    rental_id INT NULL,  
    problem_description VARCHAR(500) NOT NULL,  
    PRIMARY KEY (audit_id)  
);
```

```
DELIMITER $$
```

```
CREATE TRIGGER trg_rental_before_insert  
BEFORE INSERT ON rental  
FOR EACH ROW  
BEGIN  
    DECLARE qtd_alugueis_abertos INT DEFAULT 0;  
  
    SELECT COUNT(*)  
    INTO qtd_alugueis_abertos  
    FROM rental  
    WHERE customer_id = NEW.customer_id  
    AND return_date IS NULL;  
  
    IF qtd_alugueis_abertos > 0 THEN  
  
        INSERT INTO rental_audit_log (  
            db_user,  
            operation,  
            customer_id,  
            rental_id,  
            problem_description  
        )
```

```
VALUES (  
    CURRENT_USER(),  
    'INSERT',  
    NEW.customer_id,  
    NULL,  
    'Tentativa de realizar novo aluguel enquanto o cliente ainda possui um aluguel em  
aberto.'  
);
```

```
SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT =  
'OPERACAO BLOQUEADA: o cliente possui um aluguel em aberto.';
```

```
END IF;  
END$$
```

```
DELIMITER ;
```

```
DELIMITER $$
```

```
CREATE TRIGGER trg_rental_before_update  
BEFORE UPDATE ON rental  
FOR EACH ROW  
BEGIN  
    DECLARE qtd_alugueis_abertos INT DEFAULT 0;
```

```
IF NEW.return_date IS NULL THEN
```

```
    SELECT COUNT(*)  
    INTO qtd_alugueis_abertos  
    FROM rental  
    WHERE customer_id = NEW.customer_id  
        AND return_date IS NULL  
        AND rental_id <> OLD.rental_id;
```

```
IF qtd_alugueis_abertos > 0 THEN
```

```
    INSERT INTO rental_audit_log (
```

```

        db_user,
        operation,
        customer_id,
        rental_id,
        problem_description
    )
VALUES (
    CURRENT_USER(),
    'UPDATE',
    NEW.customer_id,
    NEW.rental_id,
    'Tentativa de criar um segundo aluguel em aberto para o mesmo cliente.'
);

SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT =
'OPERACAO BLOQUEADA: o cliente ja possui outro aluguel em aberto.';

END IF;

END IF;
END$$

DELIMITER ;

```

Questão 4

```

DELIMITER $$

DROP PROCEDURE IF EXISTS top10FilmesMaisAlugadosPorLojaECategoria$$

CREATE PROCEDURE top10FilmesMaisAlugadosPorLojaECategoria(

```

```

    IN p_categoria VARCHAR(100),
    IN p_loja_id INT
)
BEGIN
    -- Lança um erro se a categoria estiver vazia
    IF p_categoria IS NULL OR TRIM(p_categoria) = " THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'O campo de categoria está vazio!' ;
    END IF;

    -- Lança um erro se a categoria não existir
    IF NOT EXISTS (
        SELECT 1 FROM category
        WHERE name LIKE CONCAT('%', p_categoria, '%')
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Essa categoria não existe!' ;
    END IF;

    -- Lança um erro caso a loja não exista na tabela store
    IF NOT EXISTS (SELECT 1 FROM store WHERE store_id = p_loja_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = "Essa loja não existe!";
    END IF;

    -- Consulta
    SELECT
        f.title AS filme,
        COUNT(r.rental_id) AS total_alugueis,
        p_categoria AS categoria,
        i.store_id AS id_loja
    FROM rental r
    INNER JOIN inventory i ON r.inventory_id = i.inventory_id
    INNER JOIN film f ON i.film_id = f.film_id
    INNER JOIN film_category fc ON f.film_id = fc.film_id
    INNER JOIN category c ON fc.category_id = c.category_id
    WHERE c.name LIKE CONCAT('%', p_categoria, '%') AND i.store_id = p_loja_id
    GROUP BY f.film_id, f.title
    ORDER BY total_alugueis DESC, filme ASC
    LIMIT 10;
END$$

DELIMITER ;

```

Explicação:

Uma procedure é um conjunto de instruções SQL reutilizável, que pode receber parâmetros para sua execução. O exemplo acima é uma procedure que lista o Top 10 filmes mais alugados de uma determinada categoria e loja, passadas via parâmetro.

Antes da execução do SELECT, há três verificações:

- 'O parâmetro de categoria está vazio?'
- 'Essa categoria existe na tabela de categorias?' e
- 'Existe alguma loja com o id passado por parâmetro?'

Caso o processo caia em alguma dessas validações, a execução é interrompida e é lançado um erro de código '45000', número utilizado para representar erros não catalogados pela linguagem SQL.

Para a consulta SELECT, é utilizada a tabela 'rental', fazendo o seguinte encadeamento de junções (JOINS):

'rental' JOIN 'inventory' JOIN 'film' JOIN 'film_category' JOIN 'category'.

Para o total de alugueis é utilizado o agregador COUNT(r.rental_id) que conta quantas vezes um filme foi alugado.

A cláusula WHERE utiliza o LIKE para filtrar filmes que possuem em uma de suas categorias o nome passado por parâmetro, que está entre '%' para buscar o termo em qualquer posição da STRING. Também são filtrados os registros por id da loja.

São condensada todas as repetições de um filme em uma única linha com " GROUP BY f.film_id, f.title" e ordenados os registros descendente pelo total de alugueis, e como segundo critério, ascendentemente por título do filme.

Por fim, a consulta é limitada até 10 registros, já que a procedure é um ranking TOP 10.

Questão 5

View para o perfil gerencial

vw_painel_gerencial

A View gerencial tem como objetivo apresentar informações resumidas sobre os alugueis realizados na locadora.

Código SQL

```
CREATE VIEW vw_painel_gerencial AS
```

```
SELECT  
    cat.name AS categoria,
```

```

COUNT(r.rental_id) AS quantidade_alugueis,
SUM(p.amount) AS receita_total,
AVG(p.amount) AS ticket_medio

FROM rental r

JOIN inventory i
  ON r.inventory_id = i.inventory_id

JOIN film f
  ON i.film_id = f.film_id

JOIN film_category fc
  ON f.film_id = fc.film_id

JOIN category cat
  ON fc.category_id = cat.category_id

JOIN payment p
  ON r.rental_id = p.rental_id

GROUP BY cat.name;

```

Funcionamento

A consulta utiliza o JOIN para relacionar as tabelas necessárias do banco Sakila.

A tabela **rental** contém os registros dos alugueis. Através dela é possível chegar ao estoque (**inventory**), ao filme (**film**) e à categoria (**category**).

A tabela **payment** é utilizada para obter os valores pagos pelos alugueis.

A função **COUNT()** calcula a quantidade de alugueis, **SUM()** calcula a receita total e **AVG()** calcula o valor médio dos pagamentos.

O **GROUP BY** agrupa os resultados por categoria.

View para o perfil analítico

vw_alugueis_detalhados

A View analítica tem como objetivo apresentar os detalhes dos alugueis realizados.

Código SQL

```
CREATE VIEW vw_alugueis_detalhados AS
```

```
SELECT
    r.rental_date AS data_aluguel,
    c.first_name AS nome_cliente,
    c.last_name AS sobrenome_cliente,
    f.title AS filme,
    cat.name AS categoria,
    p.amount AS valor

FROM rental r

JOIN customer c
    ON r.customer_id = c.customer_id

JOIN inventory i
    ON r.inventory_id = i.inventory_id

JOIN film f
    ON i.film_id = f.film_id

JOIN film_category fc
    ON f.film_id = fc.film_id

JOIN category cat
    ON fc.category_id = cat.category_id

JOIN payment p
    ON r.rental_id = p.rental_id;
```

Funcionamento

A consulta utiliza a tabela **rental** como ponto de partida e realiza **JOINS** com outras tabelas para reunir as informações necessárias.

A tabela **customer** permite identificar o cliente que realizou o aluguel.

A tabela **inventory** permite identificar qual exemplar foi alugado e, através dela, chegar à tabela **film**, que contém as informações do filme.

As tabelas **film_category** e **category** permitem identificar a categoria do filme.

Por fim, a tabela **payment** permite obter o valor pago relacionado ao aluguel.

Diferentemente da View gerencial, os registros não são agrupados, pois o objetivo é apresentar os detalhes das operações.
